

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Ứng dụng Mô hình ngôn ngữ lớn và Tác tử AI trong
việc xây dựng hệ thống hỗ trợ học tập

NGUYỄN KHẮC ĐIỆP

diep.nk225806@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: TS. Ngô Văn Linh

Ngành: Công nghệ thông tin

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Ứng dụng Mô hình ngôn ngữ lớn và Tác tử AI trong
việc xây dựng hệ thống hỗ trợ học tập

NGUYỄN KHẮC ĐIỆP

diep.nk225806@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: TS. Ngô Văn Linh _____

Chữ kí GVHD

Khoa: Công nghệ thông tin Việt-Nhật

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2022

LỜI CẢM ƠN

Em tôi xin gửi lời cảm ơn sâu sắc tới thầy (cô) hướng dẫn đã kiên nhẫn hướng dẫn, đưa ra những ý kiến quý báu, giúp em hoàn thành đề tài này. Em cũng xin cảm ơn các thầy cô giảng dạy khóa, những người đã truyền đạt kiến thức nền tảng trong lĩnh vực công nghệ. Đặc biệt, em xin cảm ơn gia đình và bạn bè đã luôn động viên, ủng hộ, tạo điều kiện thuận lợi để em tập trung hoàn thành đồ án này. Những đóng góp quý báu từ mỗi người đã giúp em vượt qua những khó khăn và hoàn thành công việc.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Hệ thống giáo dục hiện đại đòi hỏi các công cụ hỗ trợ học tập thông minh, nhất là sau đại dịch COVID-19. Các chatbot AI tổng quát như ChatGPT, mặc dù mạnh mẽ, nhưng dễ bịa chuyện (hallucination) và không được đặc thù hóa cho sách giáo khoa Việt Nam. Các hệ thống Intelligent Tutoring System (ITS) hiệu quả nhưng thiếu phiên bản tiếng Việt, còn Learning Management System (LMS) cung cấp nền tảng lưu trữ nhưng thiếu khả năng sinh câu hỏi tự động và chấm điểm thông minh.

Đề tài này đề xuất kiến trúc **Code-level Orchestrator** kết hợp **Adaptive RAG** (Retrieval-Augmented Generation) để xây dựng hệ thống Chatbot hỗ trợ dạy-học Tin Học THPT (lớp 10-12). Phương pháp RAG được lựa chọn vì không yêu cầu fine-tune LLM toàn bộ, chi phí thấp, và dễ cập nhật khi sách giáo khoa thay đổi. Hệ thống cung cấp 4 chức năng chính: (1) trả lời câu hỏi dựa trên kiến thức từ sách giáo khoa, (2) sinh câu hỏi tự động bốn dạng (trắc nghiệm, tự luận, điền khuyết, đúng/sai), (3) chấm điểm tự động dựa trên ngữ cảnh (không chỉ so khớp từ khóa), và (4) tạo bài giảng từ nội dung sách.

Các đóng góp chính gồm: (i) xây dựng Orchestrator 100% từ code Python thuần (tránh dùng framework như LangChain/CrewAI), giúp dễ kiểm soát và giải thích; (ii) chiến lược Adaptive RAG với 4 phương pháp tìm kiếm tối ưu theo loại truy vấn; (iii) hệ thống sinh câu và chấm điểm dựa ngữ cảnh với 4 metrics đánh giá RAG (Faithfulness 0.9757, Answer Relevancy 0.8571, Context Recall 0.9593); (iv) quản lý session cộng tác hóa cho học sinh. Kết quả cho thấy hệ thống đạt độ chính xác cao và đáng tin cậy, sẵn sàng triển khai trong lớp học thực tế từ lớp 10-12.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

ABSTRACT

Mục này khuyến khích sinh viên viết lại mục “Tóm tắt” đề án tốt nghiệp ở trang trước bằng tiếng Anh. Phần này phải có đầy đủ các nội dung như trong phần tóm tắt bằng tiếng Việt. Sinh viên không nhất thiết phải trình bày mục này.

Nhưng nếu lựa chọn trình bày, sinh viên cần đảm bảo câu từ và ngữ pháp chuẩn tắc, nếu không sẽ có tác dụng ngược, gây phản cảm.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	1
1.3 Định hướng giải pháp.....	2
1.4 Bố cục đồ án	3
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....	4
2.1 Khảo sát hiện trạng	4
2.2 Tổng quan chức năng	4
2.2.1 Biểu đồ use case tổng quát	4
2.2.2 Biểu đồ use case phân rã XYZ	4
2.2.3 Quy trình nghiệp vụ	4
2.3 Đặc tả chức năng	5
2.3.1 Đặc tả use case A.....	5
2.3.2 Đặc tả use case B	5
2.4 Yêu cầu phi chức năng	5
CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG.....	6
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG	7
4.1 Môi trường và dữ liệu thực nghiệm.....	7
4.1.1 Môi trường phát triển	7
4.1.2 Dữ liệu thực nghiệm.....	7
4.2 Tổng quan kiến trúc toàn bộ hệ thống	7
4.3 Thiết kế chi tiết các thành phần hệ thống.....	10
4.3.1 Thiết kế Lớp Ngữ cảnh và Ý định (Context & Intent Layer)	10

4.3.2 Thiết kế Lớp Lập kế hoạch (Planning Layer).....	12
4.3.3 Thiết kế Lớp Thực thi (Execution Layer)	13
4.3.4 Thiết kế Lớp Tri thức (Knowledge Layer)	14
4.3.5 Thiết kế Lớp Sinh phản hồi và Hậu kiểm (Generation & Validation).....	15
4.3.6 Thiết kế Streaming Responser, History Memory và Logging	16
4.4 Đánh giá hiệu năng và Kết quả thực nghiệm.....	17
4.4.1 Phương pháp đánh giá	17
4.4.2 Kết quả thực nghiệm phân hệ RAG	18
4.4.3 Đánh giá độ trễ hệ thống (Latency).....	19
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT	20
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	21
6.1 Kết luận	21
6.2 Hướng phát triển.....	21
TÀI LIỆU THAM KHẢO.....	24
PHỤ LỤC.....	26
A. HƯỚNG DẪN VIẾT ĐỒ ÁN TỐT NGHIỆP	26
A.1 Ngành học	27
A.2 Đánh dấu (bullet) và đánh số (numering)	27
A.3 Cách thêm bảng	28
A.4 Chèn hình ảnh	28
A.5 Tài liệu tham khảo	29
A.6 Cách viết phương trình và công thức toán học.....	29
A.7 Qui cách đóng quyển.....	29
B. ĐẶC TẢ USE CASE.....	31
B.1 Đặc tả use case “Thông kê tình hình mượn sách”	31

B.2 Đặc tả use case “Đăng ký làm thẻ mượn”	31
--	----

DANH MỤC HÌNH VẼ

Hình 4.1	Sơ đồ tổng quan kiến trúc hệ thống (System Overview Architecture)	9
Hình 4.2	Chi tiết cấu trúc và luồng xử lý tại Lớp Ngữ cảnh và Ý định . .	11
Hình 4.3	Chu trình ra quyết định và duy trì ngữ cảnh tại Lớp Lập kế hoạch (Planning Layer)	12
Hình 4.4	Sơ đồ vòng lặp Đa tác vụ (Multi-Action Loop) tại Lớp Thực thi	13
Hình 4.5	Cấu trúc xử lý và truy xuất thông tin của Knowledge Layer . .	14
Hình 4.6	Giai đoạn phân luồng Tạo sinh và Hậu kiểm (Block 5)	16
Hình 4.7	Khối thao tác Phản hồi và Khắc ghi tiến trình (Block 6)	17
Hình A.1	Internet vạn vật	28
Hình A.2	Quy cách đóng quyển đồ án	30
Hình A.3	Quy cách đóng quyển đồ án	30

DANH MỤC BẢNG BIỂU

Bảng 4.1	Kết quả đánh giá hệ thống theo framework Ragas	18
Bảng 4.2	Thống kê thời gian trễ của các khối kiến trúc trong hệ thống .	19
Bảng A.1	Table to test captions and labels.	28

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
API	Giao diện lập trình ứng dụng (Application Programming Interface)
EUD	Phát triển ứng dụng người dùng cuối(End-User Development)
GWT	Công cụ lập trình Javascript bằng Java của Google (Google Web Toolkit)
HTML	Ngôn ngữ đánh dấu siêu văn bản (HyperText Markup Language)
IaaS	Dịch vụ hạ tầng

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Giáo dục hiện đại đòi hỏi sự hỗ trợ học tập cá nhân hóa và tự động hóa các tác vụ giáo dục phức hợp. Các hệ thống truyền thống như tìm kiếm thông tin hoặc câu hỏi-trả lời đơn giản không thể vượt qua hiện tượng "ảo giác" (hallucination) của các mô hình ngôn ngữ lớn và thiếu độ chính xác trong ngữ cảnh giáo dục.

Ở cấp bậc THPT, nhu cầu hỗ trợ học tập ngày càng cấp thiết. Học sinh cần một trợ lý ảo có thể trả lời câu hỏi dựa trên tài liệu chính thức như sách giáo khoa, tự động sinh ra bài tập với mức độ tùy chỉnh, chấm điểm và giải thích lỗi một cách chính xác. Đồng thời, giáo viên phải đầu tư nhiều thời gian cho việc tạo bài tập, chấm điểm, và giảng dạy lặp lại nội dung. Các hệ thống AI chung ngoài thị trường thường không tuân thủ chương trình giáo dục Việt Nam và dễ sinh ra thông tin sai lệch, làm giảm hiệu quả dạy-học.

Vấn đề cần giải quyết là xây dựng một hệ thống trợ lý ảo thông minh có khả năng tích hợp nhiều chức năng giáo dục (trả lời câu hỏi, sinh đề bài, chấm điểm, tạo tài liệu) mà vẫn đảm bảo tính chính xác cao dựa trên sách giáo khoa, giảm bớt gánh nặng cho giáo viên và nâng cao chất lượng học tập cho học sinh tại các trường THPT.

1.2 Mục tiêu và phạm vi đề tài

Hiện nay, các giải pháp hỗ trợ giáo dục AI đã tồn tại trên thị trường. Các hệ thống như ChatGPT hoặc Gemini cung cấp khả năng tương tác linh hoạt nhưng dễ sinh ra thông tin không chính xác khi áp dụng cho bối cảnh giải pháp cụ thể. Các hệ thống quản lý lớp học truyền thống (Moodle, Google Classroom) cung cấp công cụ quản lý tập trung nhưng thiếu khả năng AI tự động. Các nền tảng hỗ trợ giáo dục AI chuyên biệt từ các nước ngoài thường xây dựng cho nền tảng giáo dục quốc tế, không tối ưu cho chương trình giáo dục Việt Nam và yêu cầu chi phí cao.

Các hạn chế của các giải pháp hiện có là thiếu tính "gắn kết dữ liệu" (grounding) với tài liệu chính thức, dẫn tới sai lệch nội dung. Hầu hết các hệ thống chỉ hỗ trợ một hoặc hai chức năng đơn lẻ, không cung cấp quy trình giảng dạy toàn diện. Đặc biệt, không có cơ chế tự kiểm tra (self-reflection) chất lượng của phản hồi, dẫn tới độ tin cậy thấp.

Mục tiêu chính của đề án là xây dựng một hệ thống tên gọi "Intelligent Educational Assistant" (Trợ lý Ảo Thông Minh Hỗ Trợ Giáo Dục) với bốn chức năng cốt lõi: (i) trả lời câu hỏi (Q&A) dựa trên sách giáo khoa Cánh Diều và Kết Nối

Tri Thức (lớp 10, 11, 12), (ii) sinh tự động đề bài tập với nhiều định dạng (trắc nghiệm, điền khuyết, tự luận, đúng/sai) có độ khó tùy chỉnh, (iii) chấm điểm tự động và cung cấp giải thích lỗi dựa trên ngữ cảnh chính thức, (iv) sinh tài liệu học tập như bài giảng hoặc bản tóm tắt từ nội dung sách giáo khoa.

Phạm vi của đề án bao gồm xử lý dữ liệu từ hai bộ sách giáo khoa (Cánh Diều và Kết Nối Tri Thức) môn Tin học lớp 10, 11, 12, số lượng tương ứng là 2.348 đoạn dữ liệu sau khi xử lý theo cấu trúc phân tầng. Hệ thống tập trung vào bốn chức năng chính đã nêu, với đối tượng người dùng là học sinh và giáo viên bậc THPT.

1.3 Định hướng giải pháp

Hệ thống sử dụng kiến trúc Retrieval-Augmented Generation (RAG) kết hợp với Multi-Agent Orchestration để giải quyết các thách thức đã nêu. RAG là một phương pháp giảm hiện tượng ảo giác bằng cách truy xuất các đoạn dữ liệu liên quan từ kho tài liệu (sách giáo khoa), sau đó cho mô hình ngôn ngữ tổng hợp phản hồi dựa trên ngữ cảnh chính thức này.

Giải pháp được xây dựng từ ba thành phần chính. Thứ nhất, thành phần truy xuất thông tin (Retrieval) sử dụng "Hybrid Search" kết hợp tìm kiếm từ khóa (Custom BM25 viết từ đầu) và tìm kiếm ngữ nghĩa (Cosine Similarity trên embedding vector). Thứ hai, thành phần tổng hợp phản hồi (Generation) sử dụng Google Gemini 2.5 làm mô hình ngôn ngữ chính, kết hợp với các "Specialist Handlers" để xử lý từng loại tác vụ cụ thể. Thứ ba, thành phần tự kiểm tra (Self-Reflection) chạy một bộ hợp thức (Validator Agent) để đảm bảo phản hồi tuân thủ ngữ cảnh gốc và đáp ứng yêu cầu.

Lý do lựa chọn RAG là phương pháp này đã chứng minh hiệu quả trong việc giảm ảo giác và nâng cao độ tin cậy của mô hình ngôn ngữ. Custom BM25 được lựa chọn vì kho dữ liệu nhỏ (2.348 đoạn), không cần công cụ tối ưu hóa phức tạp như FAISS, và dễ giải thích trong báo cáo. Việc sử dụng cả tìm kiếm từ khóa và ngữ nghĩa đảm bảo phủ phát toàn diện các loại truy vấn khác nhau của người dùng. Google Gemini 2.5 được chọn vì hiệu năng cao, API ổn định, và chi phí hợp lý so với các mô hình khác.

Các đóng góp chính của đề án bao gồm: (i) xây dựng một hệ thống giáo dục tích hợp đầu tiên kết hợp bốn chức năng thành một nền tảng duy nhất, (ii) thiết kế một pipeline RAG thích ứng (Adaptive RAG) lựa chọn động chiến lược truy xuất dựa trên loại truy vấn, (iii) kiến trúc Multi-Agent Orchestration linh hoạt cho phép mở rộng chức năng dễ dàng mà không ảnh hưởng tới các thành phần hiện có, (iv) cô lập hiện tượng ảo giác của mô hình thông qua cơ chế tự kiểm tra (Self-Reflection), (v) đo lường định lượng chất lượng hệ thống sử dụng RAGAS evaluation frame-

work với bốn chỉ số (Answer Relevancy, Faithfulness, Context Precision, Context Recall).

1.4 Bố cục đề án

Báo cáo đề án tốt nghiệp này được tổ chức thành sáu chương chính như sau.

Chương 2 trình bày các nền tảng lý thuyết cơ bản hỗ trợ cho việc thiết kế hệ thống. Các khái niệm Retrieval-Augmented Generation, Multi-Agent Architecture, các mô hình embedding tiếng Việt, và các kỹ thuật xử lý ngôn ngữ tự nhiên liên quan được giới thiệu chi tiết. Chương này tạo nên cơ sở lý thuyết vững chắc cho các quyết định thiết kế trong các chương tiếp theo.

Chương 3 mô tả chi tiết các công nghệ và phương pháp được sử dụng trong hệ thống. Các thành phần kỹ thuật được trình bày bao gồm Custom BM25 search engine viết từ đầu, tìm kiếm ngữ nghĩa sử dụng mô hình embedding tiếng Việt, tìm kiếm kết hợp (Hybrid Search) với bình thường hóa Reciprocal Rank Fusion (RRF), tái xếp hạng kết quả (Cross-Encoder Reranking), và chiến lược phân chia tài liệu theo cấu trúc phân tầng (Hierarchical Chunking). Mỗi phương pháp được phân tích về lợi ích, hạn chế, và lý do lựa chọn.

Chương 4 trình bày kiến trúc tổng thể của hệ thống và quá trình triển khai. Các thành phần chính bao gồm Intent Detection pipeline để phân loại ý định người dùng, Adaptive RAG Agent để lựa chọn chiến lược truy xuất phù hợp, Specialist Handlers cluster để xử lý từng tác vụ cụ thể (Chat, Quiz Generation, Answer Scoring, Content Generation), và cơ chế Self-Reflection đảm bảo chất lượng. Phần này cũng mô tả cách các module tương tác với nhau thông qua một pipeline xuyên suốt.

Chương 5 trình bày các đóng góp chính của đề án cùng với kết quả thực nghiệm. Phần này bao gồm mô tả giao diện người dùng được xây dựng bằng Gradio, các kết quả định lượng sử dụng RAGAS framework, và các trường hợp nghiên cứu minh họa khả năng của hệ thống trong các tình huống sử dụng thực tế.

Chương 6 nêu rõ các hạn chế hiện tại của hệ thống như độ chính xác của Intent Detection, tốc độ xử lý, và quy mô dữ liệu. Báo cáo đề xuất các hướng cải tiến trong tương lai bao gồm fine-tuning mô hình Intent Detection, triển khai cơ chế caching để tăng tốc độ, và mở rộng hệ thống sang các môn học khác. Cuối cùng, chương này cung cấp các nhận xét kết luận về tác dụng học tập của hệ thống và triển vọng phát triển lâu dài.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương này có độ dài từ 9 đến 11 trang.

Với phương pháp phân tích thiết kế hướng đối tượng, sinh viên sử dụng biểu đồ use case theo hướng dẫn của template này. Với các phương pháp khác, sinh viên trao đổi với giáo viên hướng dẫn để đổi tên và sắp xếp lại đề mục cho phù hợp. Ví dụ, thay vì sử dụng biểu đồ use case, sinh viên đi theo hướng tiếp cận Agile có thể dùng User Story.

Lưu ý: Mỗi chương nên có thêm 1 đoạn mở đầu chương và kết thúc chương, mở đầu giới thiệu những nội dung sẽ trình bày trong chương, kết thúc tổng kết lại các nội dung đã trình bày

2.1 Khảo sát hiện trạng

Thông thường, khảo sát chi tiết về hiện trạng và yêu cầu của phần mềm sẽ được lấy từ ba nguồn chính, đó là (i) người dùng/khách hàng, (ii) các hệ thống đã có, (iii) và các ứng dụng tương tự. Sinh viên cần tiến hành phân tích, so sánh, đánh giá chi tiết ưu nhược điểm của các sản phẩm/nghiên cứu hiện có. Sinh viên có thể lập bảng so sánh nếu cần thiết. Kết hợp với khảo sát người dùng/khách hàng (nếu có), sinh viên nêu và mô tả sơ lược các tính năng phần mềm quan trọng cần phát triển.

2.2 Tổng quan chức năng

Phần 2.2 này có nhiệm vụ tóm tắt các chức năng của phần mềm. Trong phần này, sinh viên lưu ý chỉ mô tả chức năng mức cao (tổng quan) mà không đặc tả chi tiết cho từng chức năng. Đặc tả chi tiết được trình bày trong phần 2.3.

2.2.1 Biểu đồ use case tổng quát

Sinh viên vẽ biểu đồ use case tổng quan và giải thích các tác nhân tham gia là gì, nêu vai trò của từng tác nhân, và mô tả ngắn gọn các use case chính.

2.2.2 Biểu đồ use case phân rã XYZ

Với mỗi use case mức cao trong biểu đồ use case tổng quan, sinh viên tạo một mục riêng như mục 2.2.2 và tiến hành phân rã use case đó. Lưu ý tên use case cần phân rã trong biểu đồ use case tổng quan phải khớp với tên đề mục.

Trong mỗi mục như vậy, sinh viên vẽ và giải thích ngắn gọn các use case phân rã.

2.2.3 Quy trình nghiệp vụ

Nếu sản phẩm/hệ thống cần xây dựng có quy trình nghiệp vụ quan trọng/đáng chú ý, sinh viên cần mô tả và vẽ biểu đồ hoạt động minh họa quy trình nghiệp vụ

đó. Sinh viên lưu ý đây không phải là luồng sự kiện của từng use case, mà là luồng hoạt động kết hợp nhiều use case để thực hiện một nghiệp vụ nào đó.

Ví dụ, một hệ thống quản lý thư viện có quy trình nghiệp vụ mượn trả với mô tả sơ bộ như sau: Sinh viên làm thẻ mượn, sau đó sinh viên đăng ký mượn sách, thủ thư cho mượn, và cuối cùng sinh viên trả lại sách cho thư viện. Một hệ thống có thể có một vài quy trình nghiệp vụ quan trọng như vậy.

2.3 Đặc tả chức năng

Sinh viên lựa chọn từ 4 đến 7 use case quan trọng nhất của đề án để đặc tả chi tiết. Mỗi đặc tả bao gồm ít nhất các thông tin sau: (i) Tên use case, (ii) Luồng sự kiện (chính và phát sinh), (iii) Tiền điều kiện, và (iv) Hậu điều kiện. Sinh viên chỉ vẽ bổ sung biểu đồ hoạt động khi đặc tả use case phức tạp.

2.3.1 Đặc tả use case A

2.3.2 Đặc tả use case B

2.4 Yêu cầu phi chức năng

Trong phần này, sinh viên đưa ra các yêu cầu khác nếu có, bao gồm các yêu cầu phi chức năng như hiệu năng, độ tin cậy, tính dễ dùng, tính dễ bảo trì, hoặc các yêu cầu về mặt kỹ thuật như về CSDL, công nghệ sử dụng, v.v.

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

Chương này có độ dài không quá 10 trang. Nếu cần trình bày dài hơn, sinh viên đưa vào phần phụ lục. Chú ý đây là kiến thức đã có sẵn; SV sau khi tìm hiểu được thì phân tích và tóm tắt lại. Sinh viên không trình bày dài dòng, chi tiết.

Với đề án ứng dụng, sinh viên để tên chương là “Nền tảng lý thuyết và Công nghệ sử dụng”. Trong chương này, sinh viên giới thiệu về các công nghệ, nền tảng lý thuyết sử dụng trong đề án. Sinh viên cũng có thể trình bày thêm nền tảng lý thuyết nào đó nếu cần dùng tới.

Với từng công nghệ/nền tảng/lý thuyết được trình bày, sinh viên phải phân tích rõ công nghệ/nền tảng/lý thuyết đó dùng để để giải quyết vấn đề/yêu cầu cụ thể nào ở Chương 2. Hơn nữa, với từng vấn đề/yêu cầu, sinh viên phải liệt kê danh sách các công nghệ/hướng tiếp cận tương tự có thể dùng làm lựa chọn thay thế, rồi giải thích rõ sự lựa chọn của mình.

Lưu ý: Nội dung ĐATN phải có tính chất liên kết, liền mạch, và nhất quán. Vì vậy, các công nghệ/thuật toán trình bày trong chương này phải khớp với nội dung giới thiệu của sinh viên ở phần trước đó.

Trong chương này, để tăng tính khoa học và độ tin cậy, sinh viên nên chỉ rõ nguồn kiến thức mình thu thập được ở tài liệu nào, đồng thời đưa tài liệu đó vào trong danh sách tài liệu tham khảo rồi tạo các tham chiếu chéo (xem hướng dẫn ở phụ lục A.7).

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

4.1 Môi trường và dữ liệu thực nghiệm

4.1.1 Môi trường phát triển

Hệ thống được xây dựng chủ yếu dựa trên ngôn ngữ lập trình Python, kết hợp cùng các thư viện xử lý toán học như Numpy và Pandas để xử lý ma trận vector nhúng. Việc truy xuất và sinh ngôn ngữ tự nhiên được thực hiện thông qua API của mô hình ngôn ngữ lớn Google Gemini 2.5 Flash, qua thư viện *google-genai*. Đối với tác vụ giải thuật đánh giá sự tương đồng, hệ thống sử dụng mô hình nhúng *dangvantuan/vietnamese-document-embedding* có chiều dài 768 chiều. Giai đoạn tái xếp hạng được đảm nhiệm bởi mô hình Cross-Encoder *AITeamVN/Vietnamese_Reranker*. Khác với các hệ thống phụ thuộc thư viện kín, logic tìm kiếm từ khoá (Custom BM25) được code tay tinh chỉnh ở mức toán học (TF, DF, IDF), tạo sự chủ động tùy biến cho đặc thù tiếng Việt.

4.1.2 Dữ liệu thực nghiệm

Nguồn dữ liệu gốc được khai thác từ toàn bộ tài liệu Sách Giáo Khoa Tin học lớp 10, 11 và 12, thuộc cả hai bộ sách Cánh Diều và Kết Nối Tri Thức. Qua quá trình tiền xử lý, văn bản được làm sạch và dán nhãn phân cấp từ chương, bài đến sâu nhất là tiểu mục. Tổng cộng, dữ liệu được phân rã thành 2348 khối dữ liệu (chunks) và có chứa thuộc tính level để biểu diễn độ sâu, phục vụ trực tiếp cho vòng phục hồi phân cấp (Hierarchical RAG).

4.2 Tổng quan kiến trúc toàn bộ hệ thống

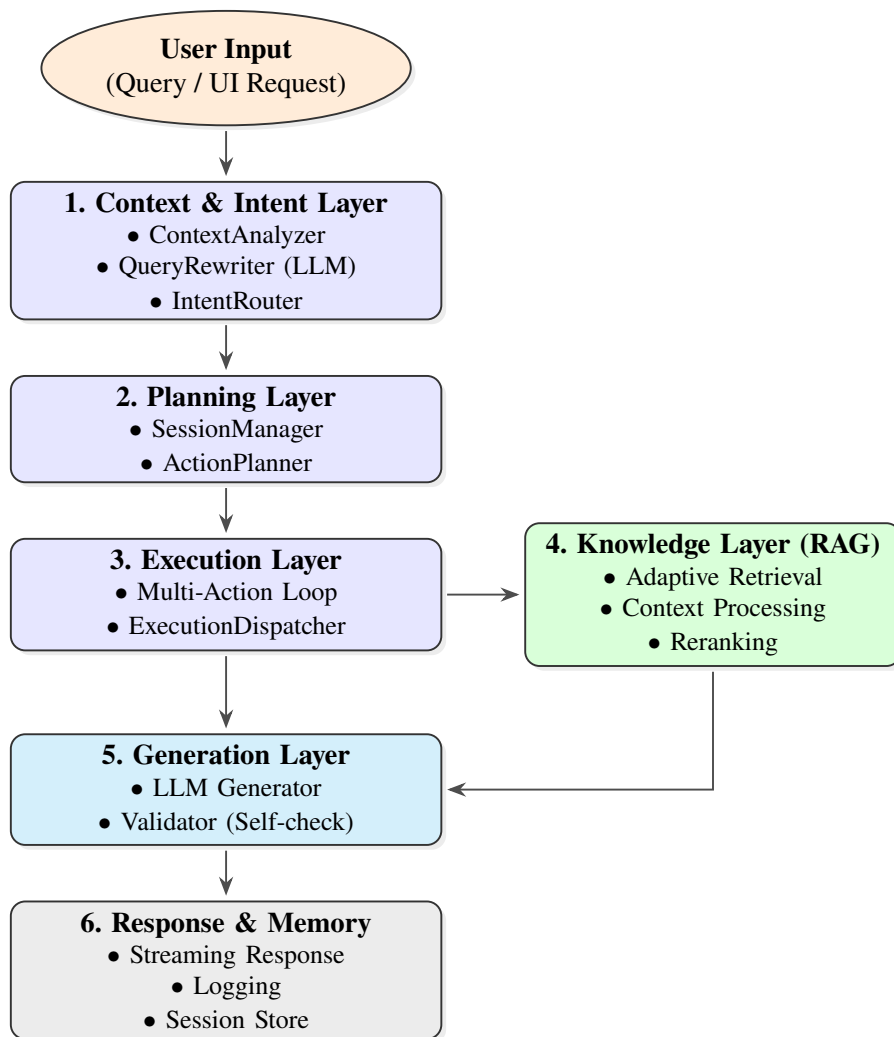
Kiến trúc tổng thể của dự án được thiết kế thành một pipeline (Data Pipeline) từ bước xử lý tài liệu thô ban đầu cho tới bước sinh câu trả lời hoàn thiện phục vụ người dùng. Quá trình này được chia làm các cụm thành phần nối tiếp nhau:

- **Cụm Tiền xử lý dữ liệu (Data Ingestion & OCR):** Nguồn dữ liệu dạng giáo trình (PDF) được nạp vào và đi qua mô hình khai phá OCR để nhận dạng, chuyển đổi văn bản sang định dạng Markdown theo chuẩn của LlamaParse. Việc này nhằm bóc tách cấu trúc phân cấp tinh vi (chương, bài, tiểu mục) của giáo trình.
- **Cụm Xây dựng Cơ sở tri thức (Knowledge Base Construction):** Văn bản Markdown sau quá trình làm sạch sẽ được phân rã (Chunking) theo đúng phân cấp. Mỗi khối dữ liệu (chunk) được nhúng (Embedding) thành ma trận vector và lưu trữ vĩnh viễn ở Cơ sở tri thức (Knowledge Base / Vector DB) cùng với

metadata (*hệ cấp và ID cha con*).

- **Cụm Phân hệ Truy xuất (Retrieval & Reranking):** Ở chặng run-time, hệ thống đón nhận truy vấn (User Query) từ người dùng. Cụm RAG làm nhiệm vụ định tuyến hướng đi, tìm kiếm không gian vector, tìm kiếm từ khoá và lai ghép (Hybrid Search + Reranker) nhằm đánh giá và rút trích các khối tri thức chứa ngữ cảnh sát nhất.
- **Cụm Sinh ngôn ngữ (LLM Generation):** Nhận Top-K ngữ cảnh cô đặc ở đầu ra của cụm Truy xuất, kết hợp cùng Prompt (câu hỏi gốc) để truyền vào mô hình ngôn ngữ lớn (Google Gemini). LLM sẽ phân tích, đối chiếu nội dung ngữ cảnh từ tri thức và sinh ra câu trả lời theo ngôn ngữ tự nhiên.

Sơ đồ tổng quan được biểu diễn tại Hình 4.1, mô tả toàn bộ quy trình xử lý từ khi tiếp nhận truy vấn của người dùng (Query / UI Request) cho đến khi sinh ra câu trả lời cuối cùng. Kiến trúc hệ thống theo mô hình tuổi thọ yêu cầu (request lifecycle) được nâng cấp và tối ưu hóa thành 6 lớp (layer) xử lý liên tục: Context & Intent Layer (Ngữ cảnh & Ý định), Planning Layer (Lập kế hoạch), Execution Layer (Thực thi), Knowledge Layer (Truy xuất Tri thức), Generation Layer (Sinh đa phương thức) và Response & Memory (Phản hồi & Ghi nhớ).



Hình 4.1: Sơ đồ tổng quan kiến trúc hệ thống (System Overview Architecture)

Trong Hình 4.1, kiến trúc hệ thống hoạt động theo các lớp liên kết chặt chẽ trong một vòng tuần hoàn khép kín:

1. Context & Intent Layer (Lớp Ngữ cảnh & Ý định): Xử lý truy vấn đầu vào, phân tích định danh lịch sử bằng *ContextAnalyzer*, viết lại câu hỏi hoặc tạo đa truy vấn (multi-queries) bằng *QueryRewriter*, và dùng *IntentRouter* để nội suy, phân nhánh ý định cốt lõi của người dùng.

2. Planning Layer (Lớp Lập kế hoạch): Đóng vai trò lớp phối hợp, hệ thống quản lý phiên làm việc thông qua *SessionManager* và định hình chiến lược/danh sách các hành động chi tiết bắt buộc phải làm nhờ bộ *ActionPlanner*.

3. Execution Layer (Lớp Thực thi): Thông qua vòng lặp *Multi-Action Loop* kết hợp cùng lõi *ExecutionDispatcher*, lớp này sẽ điều phối linh hoạt các mô-đun dịch vụ (Quiz, Slides, v.v.) và gọi lớp RAG khi nhận thấy tác vụ cần tri thức ngoài.

4. Knowledge Layer (Lớp Tri thức - RAG): Đảm nhận khâu phân giải kiến thức. Hệ thống truy xuất theo cấu hình chiến lược (*Adaptive Retrieval*), thu thập

văn bản liên quan (*Context Processing*) và sắp xếp đo lường lại mức độ liên quan thông qua mô hình chấm điểm (*Reranking*).

5. Generation Layer (Lớp Sinh nội dung): Kết hợp các kết quả thực thi và tri thức được truy xuất cô đặc vào *LLM Generator* để sinh ra đáp án và văn bản cuối cùng, có sự kiểm duyệt đối soát chéo của mô-đun *Validator* để đảm bảo chuẩn mực chất lượng.

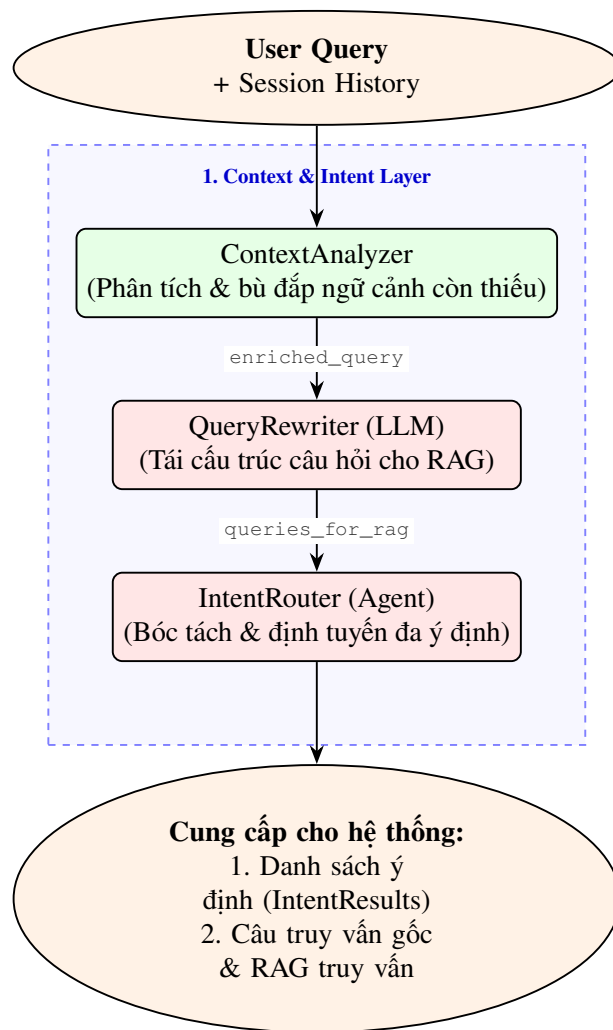
6. Response & Memory (Lớp Phản hồi & Ghi nhớ): Các kết quả cuối cùng được trả về cho người dùng qua giao thức thời gian thực (*Streaming Response*), ghi nhận lịch sử vết lỗi (*Logging*) và lưu lại dữ liệu hội thoại vào cơ sở dữ liệu hệ thống (*Session Store*) làm tài nguyên cho luồng thao tác kế tiếp.

4.3 Thiết kế chi tiết các thành phần hệ thống

Dựa trên sơ đồ kiến trúc tổng quát tại Hình 4.1, các thành phần cốt lõi của hệ thống được thiết kế chi tiết nhằm đảm bảo tính linh hoạt và độ chính xác cao trong quá trình xử lý:

4.3.1 Thiết kế Lớp Ngữ cảnh và Ý định (Context & Intent Layer)

Đây là lớp tiếp nhận và xử lý đầu vào đầu tiên của hệ thống, thực hiện nhiệm vụ bóc tách ngữ cảnh (*ContextAnalyzer*), tinh chỉnh truy vấn (*QueryRewriter*) và định tuyến ý định đa nhiệm (*IntentRouter*). Thiết kế này giúp hệ thống hiểu được không chỉ câu hỏi hiện tại mà còn cả diễn biến của toàn bộ cuộc hội thoại trước đó.



Hình 4.2: Chi tiết cấu trúc và luồng xử lý tại Lớp Ngữ cảnh và Ý định

Lớp Ngữ cảnh và Ý định hoạt động tuân theo trình tự ba bước cốt lõi:

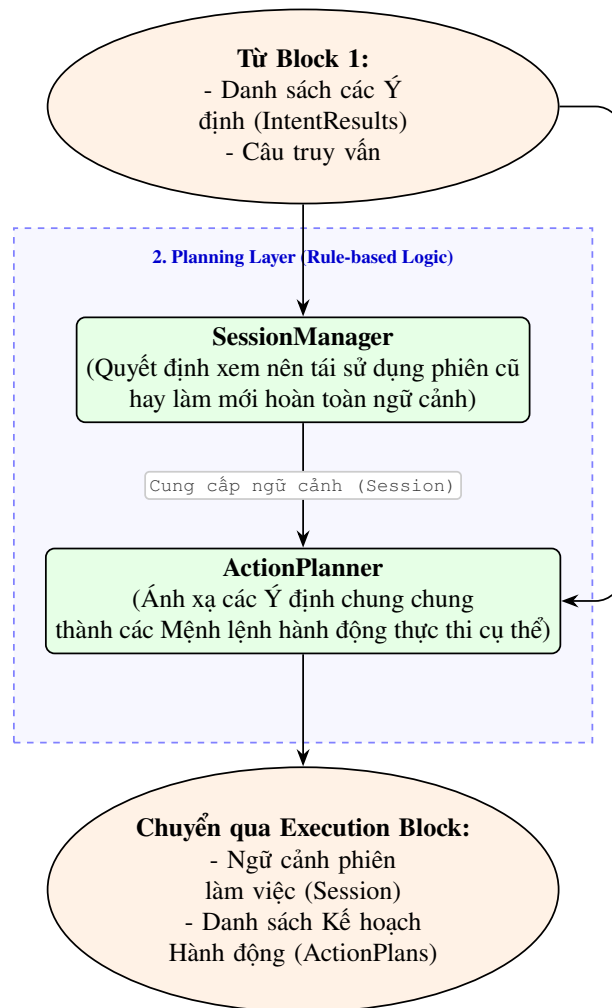
- **ContextAnalyzer (Phân tích ngữ cảnh):** Đánh giá mức độ phụ thuộc của truy vấn hiện tại vào lịch sử hội thoại trước đó (Session History). Mô-đun này được viết bằng mã nguồn logic xác định xem câu hỏi có chứa các đại từ ngữ nghĩa ẩn ý cần khôi phục ngữ cảnh hay không.
- **QueryRewriter (Viết lại truy vấn - LLM):** Khi phát hiện truy vấn không đầy đủ ngữ nghĩa hoặc hệ thống cần tìm kiếm diện rộng trong RAG, LLM được gọi để sinh ra các câu hỏi hoàn chỉnh, và có thể tạo đa truy vấn (multi-queries).
- **IntentRouter (Định tuyến ý định):** Là thành phần cốt lõi, sử dụng hàm *detect_multi()* để xác định đồng thời tối đa 3 ý định (Intent) từ người dùng. Đầu ra là một danh sách *List[IntentResult]* bao gồm siêu dữ liệu: Ý định (Intent), Loại tác vụ (Task Type), Chủ đề (Topic), Cấp bậc/Lớp (Grade), và Thứ tự ưu tiên (Order).

Nhờ kiến trúc này, hệ thống giải quyết xuất sắc các yêu cầu phức hợp (multi-intent)

trong khi vẫn duy trì đúng đường định tuyến thiết kế cho các tác vụ tiếp theo.

4.3.2 Thiết kế Lớp Lập kế hoạch (Planning Layer)

Đóng vai trò như bộ não điều phối, Lớp Lập kế hoạch (Planning Layer) chịu trách nhiệm tiếp nhận và hiện thực hóa các ý định đa nhiệm thành lệnh thực thi, thay thế cho cơ chế định tuyến thủ công tĩnh thông thường. Lớp này hoạt động hoàn toàn trên phương pháp phi xác suất dựa trên luật (Rule-based Logic) thay vì LLM, giúp đảm bảo hệ thống phản hồi cực nhanh, ổn định và mang tính chất dự đoán được.



Hình 4.3: Chu trình ra quyết định và duy trì ngữ cảnh tại Lớp Lập kế hoạch (Planning Layer)

Dữ liệu đầu vào của lớp là tập hợp các *Intent* cùng câu truy vấn nguyên thủy (Query). Lớp Lập kế hoạch phân bổ chu trình qua hai thành phần cốt lõi:

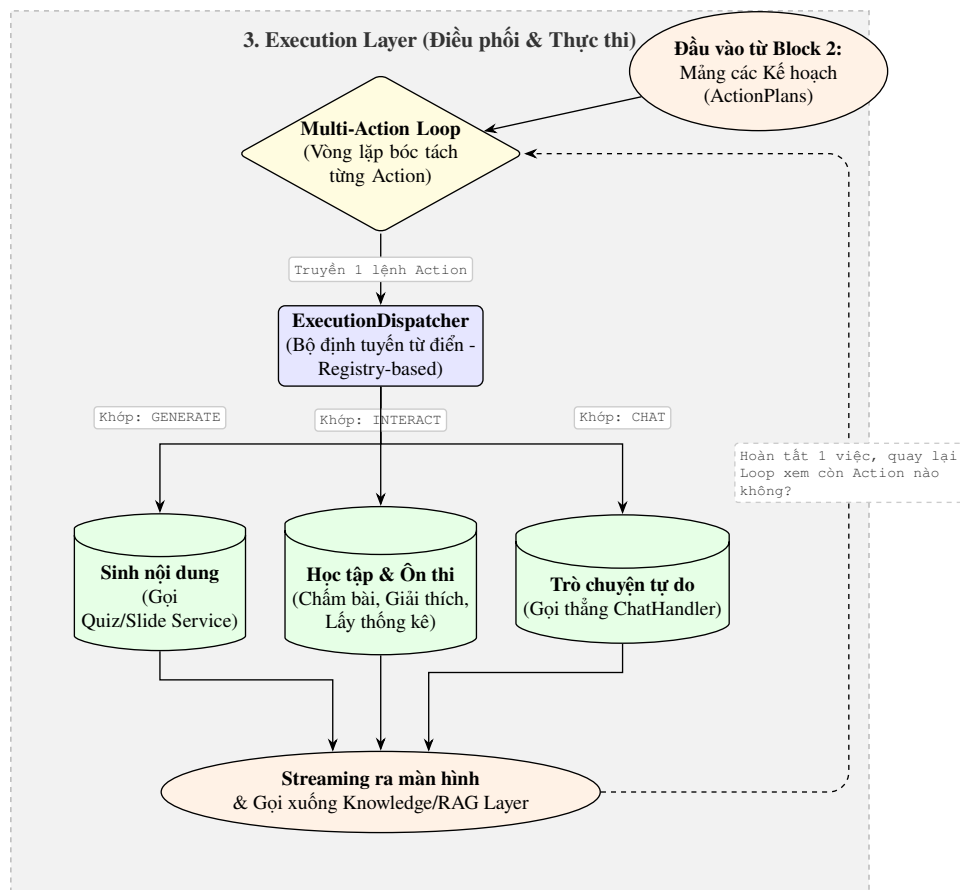
- **SessionManager:** Nhận truy vấn và xác định người dùng đang tiếp nối một chuỗi hội thoại cũ (cần tải lại ngữ cảnh quá khứ) hay khởi tạo tác vụ mới hoàn toàn. Khối này sẽ trực tiếp duy trì không gian *Session* phù hợp.

- **ActionPlanner:** Dựa trên cấu trúc Session nhận được từ SessionManager và các đặc trưng từ danh sách Intent, mô-đun này sẽ ánh xạ các ý định chung chung thành cấu trúc *ActionPlans* (Danh sách Lệnh hành động) rõ cấp bậc và không trùng lặp (deduplicated).

Đầu ra cuối cùng là danh sách tác vụ rành mạch, kết thúc vai trò nền tảng để đẩy sang Lớp Thực thi (Execution Layer).

4.3.3 Thiết kế Lớp Thực thi (Execution Layer)

Lớp Thực thi đóng vai trò trung tâm trong chuỗi xử lý đa nhiệm của hệ thống, có nhiệm vụ bóc tách danh sách các thao tác đã được Lập kế hoạch và phân phối chúng đến đúng bộ dịch vụ chuyên trách (Handler). Cơ chế định tuyến này giúp quy trình diễn ra liên tục, linh hoạt và độ chính xác cao.



Hình 4.4: Sơ đồ vòng lặp Đa tác vụ (Multi-Action Loop) tại Lớp Thực thi

Quá trình thực thi trong lớp này gồm các thành phần cốt lõi:

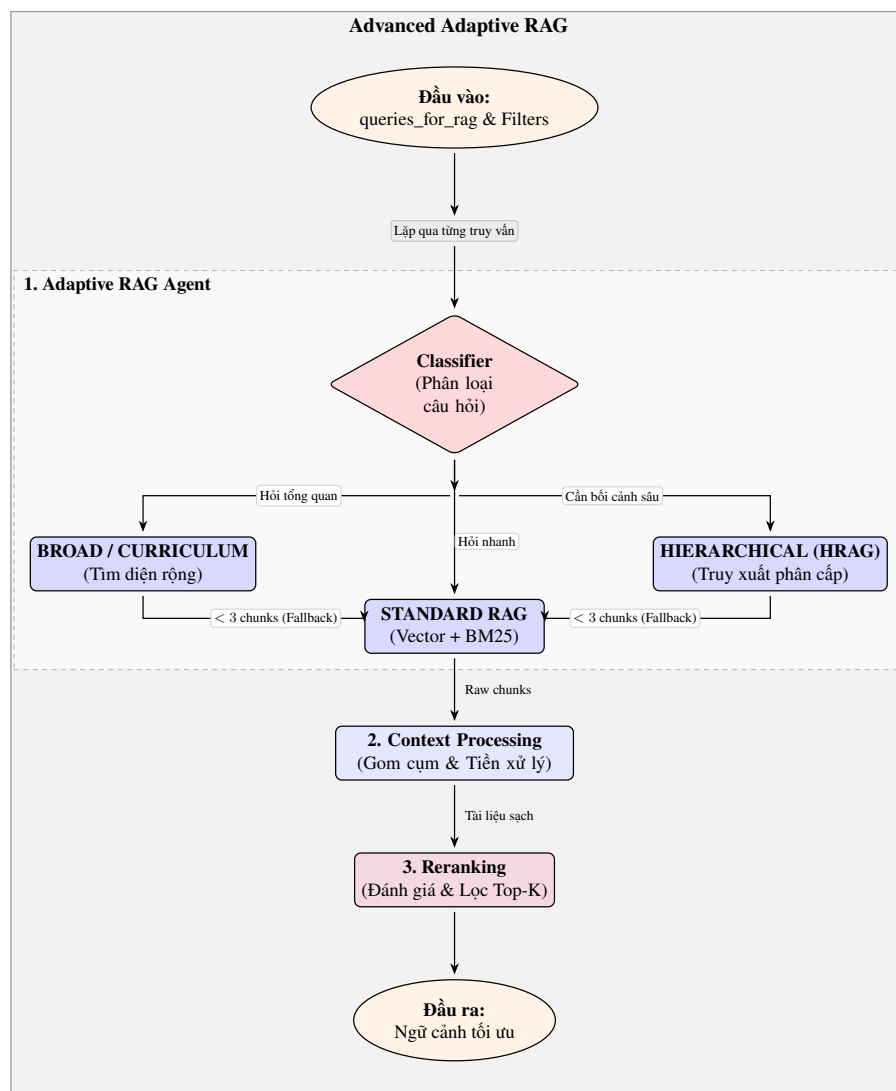
- **Multi-Action Loop:** Hệ thống sẽ duyệt qua mảng hành động (ActionPlans) theo phương thức lặp vòng. Vòng lặp đóng vai trò như một bộ đếm trạng thái, dừng lại khi tất cả Action đã được bóc tách hoàn chỉnh.
- **Execution Dispatcher:** Dựa vào khóa định danh của mỗi tác vụ, Dispatcher

sử dụng cơ chế nối kết từ điển (Registry-based) để chọn ra dịch vụ thích hợp gồm các module lớn như: Tạo Quiz/Slide, Xử lý Chấm điểm tương tác, hay Chat tự do (không cần RAG).

Kết thúc một luồng lệnh đơn lẻ, dữ liệu sẽ được trả về dạng Stream thời gian thực ra giao diện và gọi sâu xuống tầng Tri thức. Sau đó, tín hiệu sẽ vòng ngược lại Multi-Action Loop để tiếp tục gỡ lệnh kế tiếp.

4.3.4 Thiết kế Lớp Tri thức (Knowledge Layer)

Lớp Tri thức (Knowledge Layer) đóng vai trò nòng cốt trong việc giải quyết các tác vụ chuyên sâu (Knowledge-Intensive Tasks). Kiến trúc này được nâng cấp thành *Advanced Adaptive RAG*, không hoạt động trên một luồng cố định mà sử dụng Bộ phân loại (Classifier) để tự động định hướng cách thức truy xuất linh hoạt dựa trên dạng câu hỏi.



Hình 4.5: Cấu trúc xử lý và truy xuất thông tin của Knowledge Layer

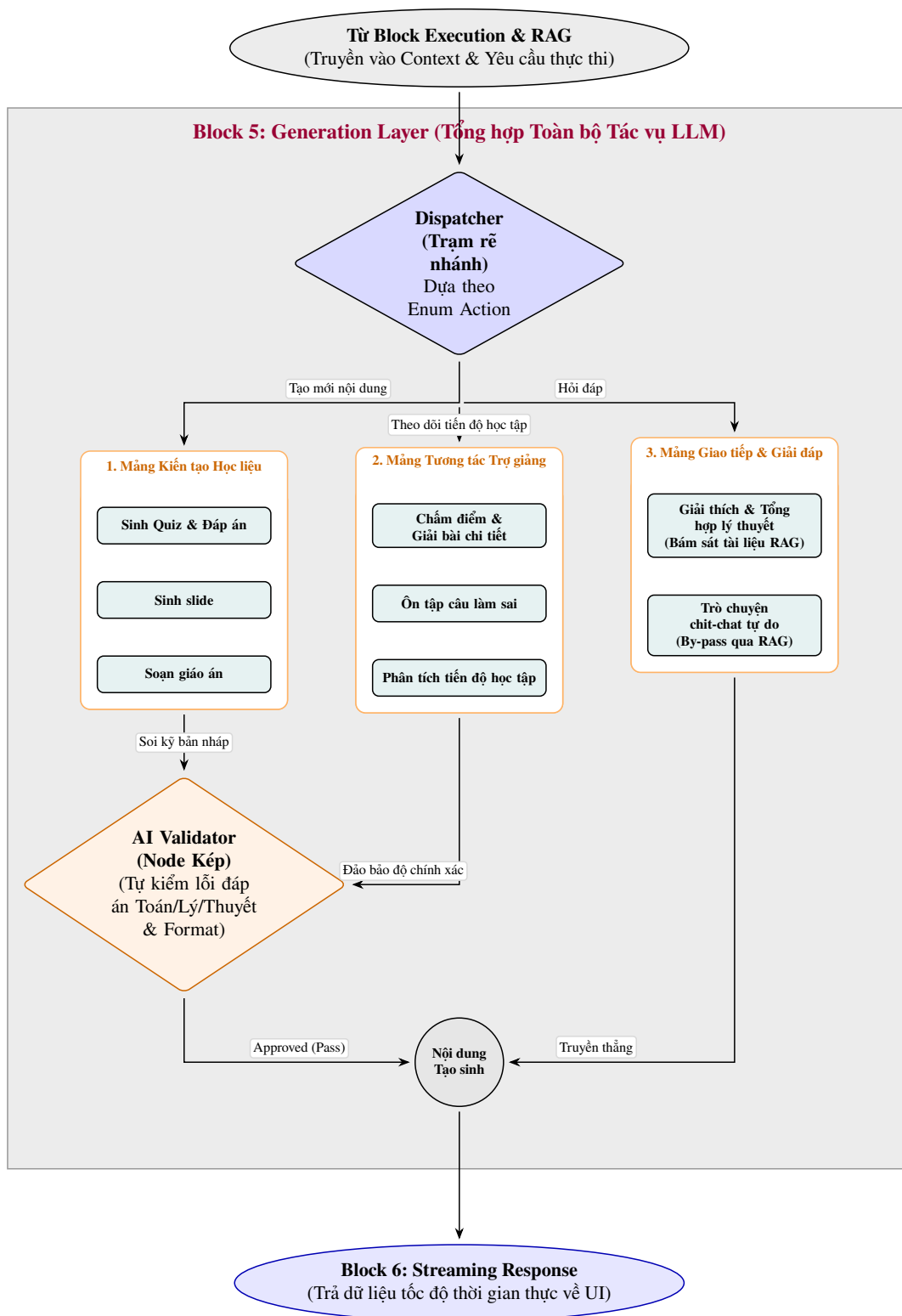
Đầu vào là mảng kết quả *queries* và các luồng lọc (filters) nhận được từ Lớp

trước đó. Quy trình chiết xuất thông tin tuân tự như sau:

- **1. Adaptive RAG Agent:** Bộ phân loại (Classifier) phân tích ý định sâu của truy vấn (hỏi nhanh, hỏi tổng quan hay cần bối cảnh sâu) để định tuyến sang dạng RAG tiêu chuẩn hoặc HRAG. Một thiết kế chốt chặn (Fallback Agent) sẽ tự động quay hồi về quy trình thông thường nếu việc rà soát trước đó rơi vào ngõ cụt (tìm được quá ít ngữ cảnh).
- **2. Context Processing:** Sau khi các Agent nhả ra đa dạng mảnh nguyên liệu (Raw chunks), quy trình sẽ gom tập trung và lọc đi các đoạn trùng lặp (Deduplication).
- **3. Reranking:** Cuối cùng, rổ tài liệu sau chuẩn hóa đi qua mô hình học máy (Cross-Encoder) để chấm điểm và tái xếp hạng. Top 5 Chunk giá trị nhất sẽ xuất ra làm thành phần ghép cốt lõi vào prompt để mô hình Generative phản hồi.

4.3.5 Thiết kế Lớp Sinh phản hồi và Hậu kiểm (Generation & Validation)

Cuối cùng, Response Generation nhận ngữ cảnh tinh lọc để biên soạn câu trả lời. Điểm khác biệt quan trọng là sự xuất hiện của Validator Agent, một lớp hậu kiểm phân luồng thực hiện đối soát câu trả lời vừa sinh ra với nguồn tri thức gốc để phát hiện các lỗi "hallucination", đảm bảo rằng mọi thông tin gửi tới người dùng đều đạt độ chính xác cao nhất (Hình 4.6).

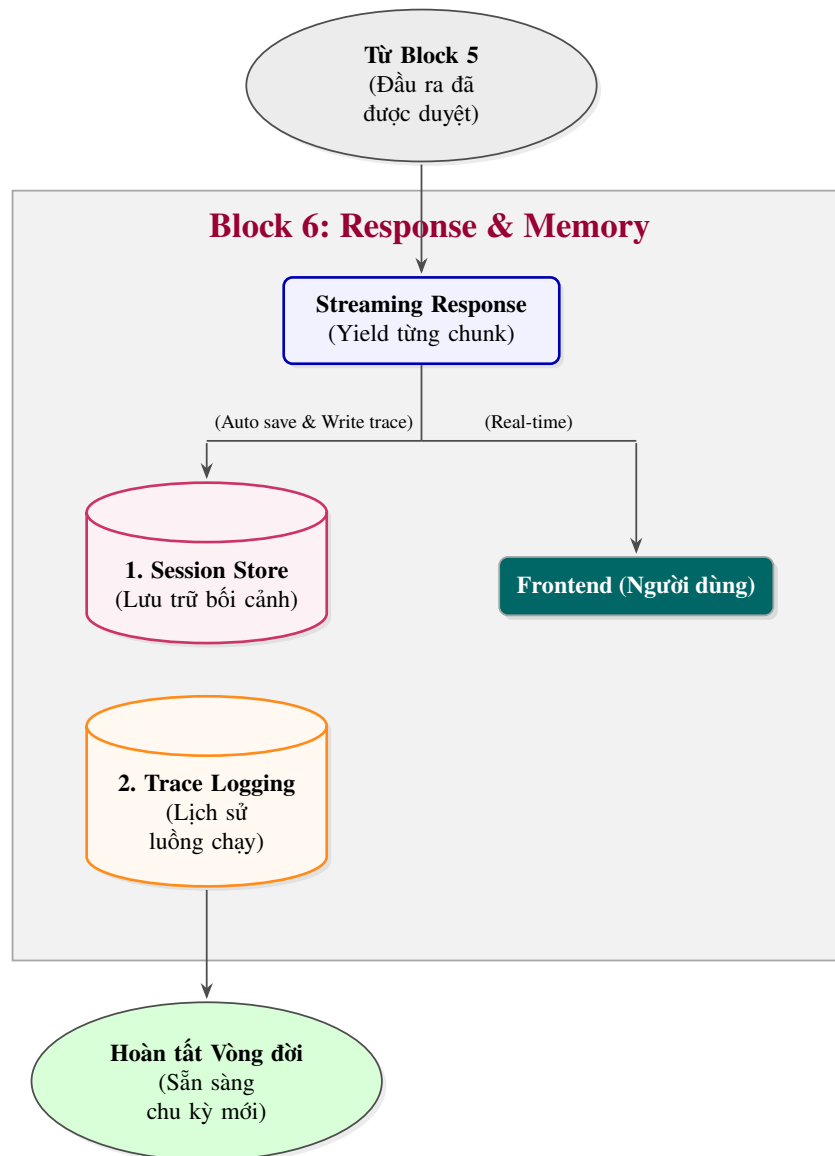


Hình 4.6: Giai đoạn phân luồng Tạo sinh và Hậu kiểm (Block 5)

4.3.6 Thiết kế Streaming Responsen, History Memory và Logging

Đây là chặng cuối cùng (Block 6) trong vòng đời xử lý của hệ thống, đóng vai trò như một cầu nối giao tiếp trực tiếp với người dùng và đồng thời dọn dẹp, lưu trữ trạng thái trước khi kết thúc phiên làm việc. Thiết kế của lớp này ưu tiên hai yếu tố: tốc độ hiển thị cho trải nghiệm người dùng (mượt mà thông qua Streaming) và

tính toàn vẹn dữ liệu của hệ thống (lưu vết và ngữ cảnh). Sơ đồ thao tác của Block 6 được thể hiện tại Hình 4.7.



Hình 4.7: Khối thao tác Phản hồi và Khắc ghi tiến trình (Block 6)

4.4 Đánh giá hiệu năng và Kết quả thực nghiệm

4.4.1 Phương pháp đánh giá

Hệ thống sử dụng phương pháp đánh giá định lượng (Quantitative Evaluation) dành riêng cho hệ thống Retrieval-Augmented Generation (RAG) thông qua Ragas framework. Tập dữ liệu kiểm thử (Testset) bao gồm 246 mẫu câu hỏi và đáp án tham chiếu (Reference) được trích xuất từ dữ liệu giáo khoa, tập trung vào nhiều cấp độ hành vi nhận thức từ Nhận biết, Thông hiểu đến Vận dụng.

Quá trình đánh giá được thực hiện trên 4 độ đo (Metrics) cốt lõi của Ragas:

- **Faithfulness (Độ trung thực):** Đo lường mức độ câu trả lời của mô hình bám

sát vào ngữ cảnh (context) mà hệ thống truy xuất được. Mức điểm cao chứng tỏ mô hình không bịa đặt thông tin (Hallucination).

- **Answer Relevancy (Độ phù hợp):** Đánh giá xem câu trả lời có giải quyết trực tiếp truy vấn ban đầu của người dùng hay không, tránh việc trả lời lan man, sai trọng tâm.
- **Context Precision (Độ chính xác ngữ cảnh):** Đánh giá khả năng xếp hạng của cụm truy xuất. Điểm số cao đồng nghĩa với việc các tài liệu liên quan nhất được xếp ở các thứ hạng đầu (Top-K).
- **Context Recall (Độ bao phủ ngữ cảnh):** Đo lường khả năng hệ thống tìm ra đầy đủ các thông tin cần thiết từ Cơ sở tri thức (Knowledge Base) so với đáp án tiêu chuẩn (Reference).

4.4.2 Kết quả thực nghiệm phân hệ RAG

Sau quá trình chạy tự động 246 mẫu thử trên toàn bộ pipeline của hệ thống, kết quả điểm số đánh giá trung bình trên các độ đo như sau:

Chỉ số đánh giá (Metric)	Điểm số trung bình (Mean)
Faithfulness (Độ trung thực)	0.9757
Answer Relevancy (Độ thiết thực của câu trả lời)	0.8571
Context Precision (Độ chính xác của ngữ cảnh)	0.8555
Context Recall (Độ bao phủ của ngữ cảnh)	0.9593

Bảng 4.1: Kết quả đánh giá hệ thống theo framework Ragas

Phân tích kết quả:

- **Faithfulness (0.9757)** và **Context Recall (0.9593)** đạt mức điểm rất cao (trên 0.95). Điều này minh chứng cho tính hiệu quả của chiến lược Hierarchical RAG kết hợp với cơ chế Chunking theo chuẩn phân cấp SGK. Hệ thống đã trích xuất được gần như hoàn hảo các đơn vị kiến thức cần thiết mà không xuất hiện tình trạng sinh thông tin ảo (Hallucination) từ phía Generative Model (Gemini). Validator Layer ở Block 5 cũng góp phần gia tăng chỉ số trung thực này.
- **Context Precision (0.8555)** và **Answer Relevancy (0.8571)** dao động ổn định ở mức 85%. Điểm số báo hiệu sự hiệu quả của mô hình tái xếp hạng Cross-Encoder trong việc đưa các tài liệu mang nhiều ý nghĩa lên đầu bảng, giúp Prompt được nhúng với ngữ cảnh làm trọng tâm, từ đó trả lời trúng đích câu hỏi người dùng.

4.4.3 Đánh giá độ trễ hệ thống (Latency)

Bên cạnh chất lượng phản hồi, hệ thống còn được đo đạc về mặt thời gian thực thi (Latency) nhằm đánh giá trải nghiệm người dùng thực tế. Các chỉ số thời gian được ghi nhận cho mỗi truy vấn đầy đủ như sau:

Thành phần thực thi	Thời gian trung bình (Giây)
Khối truy xuất (Retriever & Reranker)	4.579
Khối sinh câu trả lời (LLM Generator)	1.979
Tổng thời gian Pipeline	6.558

Bảng 4.2: Thống kê thời gian trễ của các khối kiến trúc trong hệ thống

Tổng thời gian cho một vòng lặp (từ lúc nhập truy vấn đến khi sinh mã vạch response trả về Frontend) đạt trung bình xấp xỉ 6.5 giây. Trọng số thời gian chủ yếu nằm ở khâu Retriever (hơn 4.5 giây) do hệ thống phải thực hiện Hybrid Search (bao gồm Dense Search kết hợp Custom BM25) sau đó cộng dồn với thời gian xử lý Reranker của Cross-Encoder để chất lọc dữ liệu. Tuy nhiên, việc áp dụng chiến lược sinh luồng trực tiếp (Streaming Response) ở Block 6 đã loại bỏ hiện tượng "đơ" giao diện, nâng cao trải nghiệm phản hồi (TTFT - Time to First Token được hạ thấp lý tưởng) và khiến người dùng cảm giác hệ thống trả lời tức thời.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Chương này có độ dài tối thiểu 5 trang, tối đa không giới hạn.¹ Sinh viên cần trình bày tất cả những nội dung đóng góp mà mình thấy tâm đắc nhất trong suốt quá trình làm ĐATN. Đó có thể là một loạt các vấn đề khó khăn mà sinh viên đã từng bước giải quyết được, là giải thuật cho một bài toán cụ thể, là giải pháp tổng quát cho một lớp bài toán, hoặc là mô hình/kiến trúc hữu hiệu nào đó được sinh viên thiết kế.

Chương này **là cơ sở quan trọng** để các thầy cô đánh giá sinh viên. Vì vậy, sinh viên cần phát huy tính sáng tạo, khả năng phân tích, phản biện, lập luận, tổng quát hóa vấn đề và tập trung viết cho thật tốt. Mỗi giải pháp hoặc đóng góp của sinh viên cần được trình bày trong một mục độc lập bao gồm ba mục con: (i) dẫn dắt/giới thiệu về bài toán/vấn đề, (ii) giải pháp, và (iii) kết quả đạt được (nếu có).

Sinh viên lưu ý **không trình bày lặp lại nội dung**. Những nội dung đã trình bày chi tiết trong các chương trước không được trình bày lại trong chương này. Vì vậy, với nội dung hay, mang tính đóng góp/giải pháp, sinh viên chỉ nên tóm lược/mô tả sơ bộ trong các chương trước, đồng thời tạo tham chiếu chéo tới đề mục tương ứng trong Chương 5 này. Chi tiết thông tin về đóng góp/giải pháp được trình bày trong mục đó.

Ví dụ, trong Chương 4, sinh viên có thiết kế được kiến trúc đáng lưu ý gì đó, là sự kết hợp của các kiến trúc MVC, MVP, SOA, v.v. Khi đó, sinh viên sẽ chỉ mô tả ngắn gọn kiến trúc đó ở Chương 4, rồi thêm các câu có dạng: “Chi tiết về kiến trúc này sẽ được trình bày trong phần 5.1”.

¹Trong trường hợp phần này dưới 5 trang thì sinh viên nên gộp vào phần kết luận, không tách ra một chương riêng rẽ nữa.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Sinh viên so sánh kết quả nghiên cứu hoặc sản phẩm của mình với các nghiên cứu hoặc sản phẩm tương tự.

Sinh viên phân tích trong suốt quá trình thực hiện ĐATN, mình đã làm được gì, chưa làm được gì, các đóng góp nổi bật là gì, và tổng hợp những bài học kinh nghiệm rút ra nếu có.

6.2 Hướng phát triển

Trong phần này, sinh viên trình bày định hướng công việc trong tương lai để hoàn thiện sản phẩm hoặc nghiên cứu của mình.

Trước tiên, sinh viên trình bày các công việc cần thiết để hoàn thiện các chức năng/nhiệm vụ đã làm. Sau đó sinh viên phân tích các hướng đi mới cho phép cải thiện và nâng cấp các chức năng/nhiệm vụ đã làm.

MỘT SỐ LƯU Ý VỀ TÀI LIỆU THAM KHẢO

Lưu ý: Sinh viên không được đưa bài giảng/slide, các trang Wikipedia, hoặc các trang web thông thường làm tài liệu tham khảo.

Một trang web được phép dùng làm tài liệu tham khảo **chỉ khi** nó là công bố chính thống của cá nhân hoặc tổ chức nào đó. Ví dụ, trang web đặc tả ngôn ngữ XML của tổ chức W3C <https://www.w3.org/TR/2008/REC-xml-20081126/> là TLTK hợp lệ.

Có năm loại tài liệu tham khảo mà sinh viên phải tuân thủ đúng quy định về cách thức liệt kê thông tin như sau. Lưu ý: các phần văn bản trong cặp dấu < > dưới đây chỉ là hướng dẫn khai báo cho từng loại tài liệu tham khảo; sinh viên cần xóa các phần văn bản này trong ĐATN của mình.

<**Bài báo đăng trên tạp chí khoa học:** Tên tác giả, tên bài báo, tên tạp chí, volume, từ trang đến trang (nếu có), nhà xuất bản, năm xuất bản >

[1] E. H. Hovy, "Automated discourse generation using discourse structure relations," *Artificial intelligence*, vol. 63, no. 1-2, pp. 341–385, 1993

<**Sách:** Tên tác giả, tên sách, volume (nếu có), lần tái bản (nếu có), nhà xuất bản, năm xuất bản>

[2] L. L. Peterson and B. S. Davie, *Computer networks: a systems approach*. Elsevier, 2007.

[3] N. T. Hải, *Mạng máy tính và các hệ thống mở*. Nhà xuất bản giáo dục, 1999.

<**Tập san Báo cáo Hội nghị Khoa học:** Tên tác giả, tên báo cáo, tên hội nghị, ngày (nếu có), địa điểm hội nghị, năm xuất bản>

[4] M. Poesio and B. Di Eugenio, "Discourse structure and anaphoric accessibility," in *ESSLLI workshop on information structure, discourse structure and discourse semantics*, Copenhagen, Denmark, 2001, pp. 129–143.

<**Đồ án tốt nghiệp, Luận văn Thạc sĩ, Tiến sĩ:** Tên tác giả, tên đồ án/luận văn, loại đồ án/luận văn, tên trường, địa điểm, năm xuất bản>

[5] A. Knott, "A data-driven methodology for motivating a set of coherence relations," Ph.D. dissertation, The University of Edinburgh, UK, 1996.

<**Tài liệu tham khảo từ Internet:** Tên tác giả (nếu có), tựa đề, cơ quan (nếu có), địa chỉ trang web, thời gian lần cuối truy cập trang web>

[6] T. Berners-Lee, *Hypertext transfer protocol (HTTP)*. [Online]. Available:

`ftp://info.cern.ch/pub/www/doc/http-spec.txt.Z` (visited on 09/30/2010).

[7] Princeton University, *Wordnet*. [Online]. Available: `http://www.cogsci.princeton.edu/~wn/index.shtml` (visited on 09/30/2010).

TÀI LIỆU THAM KHẢO

- [1] E. H. Hovy, “Automated discourse generation using discourse structure relations,” *Artificial intelligence*, vol. 63, no. 1-2, pp. 341–385, 1993.
- [2] L. L. Peterson and B. S. Davie, *Computer networks: a systems approach*. Elsevier, 2007.
- [3] N. T. Hải, *Mạng máy tính và các hệ thống mở*. Nhà xuất bản giáo dục, 1999.
- [4] M. Poesio and B. Di Eugenio, “Discourse structure and anaphoric accessibility,” in *ESSLLI workshop on information structure, discourse structure and discourse semantics, Copenhagen, Denmark*, 2001, pp. 129–143.
- [5] A. Knott, “A data-driven methodology for motivating a set of coherence relations,” Ph.D. dissertation, The University of Edinburgh, UK, 1996.
- [6] T. Berners-Lee, *Hypertext transfer protocol (HTTP)*. Accessed: Sep. 30, 2010. [Online]. Available: <ftp://info.cern.ch/pub/www/doc/http-spec.txt.Z>
- [7] Princeton University, *Wordnet*. Accessed: Sep. 30, 2010. [Online]. Available: <http://www.cogsci.princeton.edu/~wn/index.shtml>

PHỤ LỤC

A. HƯỚNG DẪN VIẾT ĐỒ ÁN TỐT NGHIỆP

Quy định chung

Dưới đây là một số quy định và hướng dẫn viết đồ án tốt nghiệp mà bắt buộc sinh viên phải đọc kỹ và tuân thủ nghiêm ngặt.

Sinh viên cần đảm bảo tính thống nhất toàn báo cáo (font chữ, căn dòng hai bên, hình ảnh, bảng, margin trang, đánh số trang, v.v.). Để làm được như vậy, sinh viên chỉ cần sử dụng các định dạng theo đúng template ĐATN này. Khi paste nội dung văn bản từ tài liệu khác của mình, sinh viên cần chọn kiểu Copy là “Text Only” để định dạng văn bản của template không bị phá vỡ/vi phạm.

Tuyệt đối cấm sinh viên đạo văn. Sinh viên cần ghi rõ nguồn cho tất cả những gì không tự mình viết/vẽ lên, bao gồm các câu trích dẫn, các hình ảnh, bảng biểu, v.v. Khi bị phát hiện, sinh viên sẽ không được phép bảo vệ ĐATN.

Tất cả các hình vẽ, bảng biểu, công thức, và tài liệu tham khảo trong ĐATN nhất thiết phải được SV giải thích và tham chiếu tối ít nhất một lần. Không chấp nhận các trường hợp sinh viên đưa ra hình ảnh, bảng biểu tùy hứng và không có lời mô tả/giải thích nào.

Sinh viên tuyệt đối không trình bày ĐATN theo kiểu viết ý hoặc gạch đầu dòng. ĐATN không phải là một slide thuyết trình; khi người đọc không hiểu sẽ không có ai giải thích hộ. Sinh viên cần viết thành các đoạn văn và phân tích, diễn giải đầy đủ, rõ ràng. Câu văn cần đúng ngữ pháp, đầy đủ chủ ngữ, vị ngữ và các thành phần câu. Khi thực sự cần liệt kê, sinh viên nên liệt kê theo phong cách khoa học với các ký tự La Mã. Ví dụ, nhiều sinh viên luôn cảm thấy hối hận vì (i) chưa cố gắng hết mình, (ii) chưa sắp xếp thời gian học/chơi một cách hợp lý, (iii) chưa tìm được người yêu để chia sẻ quãng đời sinh viên vất vả, và (iv) viết ĐATN một cách cẩu thả.

Trong một số trường hợp nhất thiết phải dùng các bullet để liệt kê, sinh viên cần thống nhất Style cho toàn bộ các bullet các cấp mà mình sử dụng đến trong báo cáo. Nếu dùng bullet cấp 1 là hình tròn đen, toàn bộ báo cáo cần thống nhất cách dùng như vậy; ví dụ như sau:

- Đây là mục 1 – Thực sự không còn cách nào khác tôi mới dùng đến việc bullet trong báo cáo.
- Đây là mục 2 – Nghĩ lại thì tôi có thể không cần dùng bullet cũng được. Nên tôi sẽ xóa bullet và tổ chức lại hai mục này trong báo cáo của mình cho khoa học hơn. Tôi muốn thầy cô và người đọc cảm nhận được tâm huyết của tôi

trong từng trang báo cáo ĐATN.

A.1 Ngành học

Sinh viên lưu ý viết đúng ngành/chuyên ngành trên bìa và trên gáy theo đúng quy định của Trường. Ngành học hay chuyên ngành học phụ thuộc vào ngành học mà sinh viên đăng ký. Sinh viên có thể đăng nhập trên trang quản lý học tập của mình để xem lại chính xác ngành học của mình.

Một số ví dụ sinh viên có thể tham khảo dưới đây, trong trường hợp có chuyên ngành thì sinh viên không cần ghi chuyên ngành:

1. Đối với kỹ sư chính quy:

(a) Từ K61 trở về trước: Ngành Kỹ thuật phần mềm

(b) Từ K62 trở về sau: Ngành Khoa học máy tính

2. Đối với cử nhân:

(a) Ngành Công nghệ thông tin

3. Đối với chương trình EliteTech:

(a) Chương trình Việt Nhật/KSTN: Ngành Công nghệ thông tin

(b) Chương trình ICT Global: Ngành Information Technology

(c) Chương trình DS&AI: Ngành Khoa học dữ liệu

A.2 Đánh dấu (bullet) và đánh số (numering)

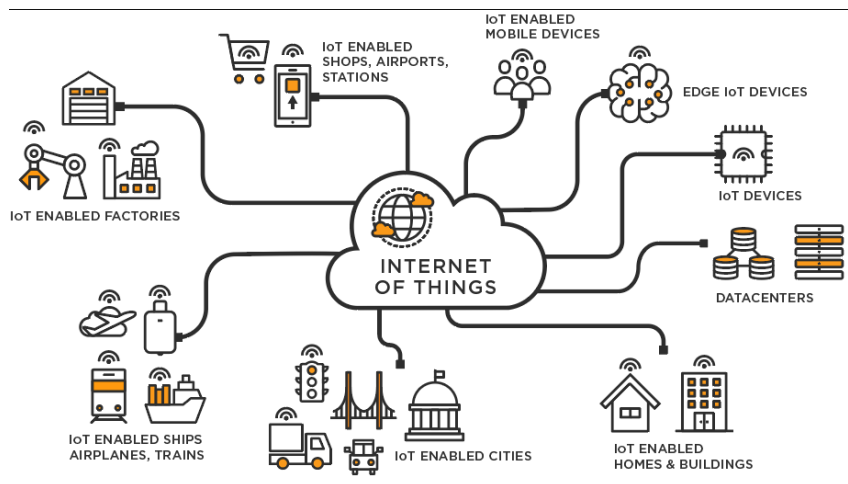
Việc sử dụng danh sách trong LaTeX khá đơn giản và không yêu cầu sinh viên phải thêm bất kỳ gói bổ sung nào. LaTeX cung cấp hai môi trường liệt kê đó là:

- Đánh dấu (bullet) là kiểu liệt kê không có thứ tự. Để sử dụng kiểu liệt kê đánh dấu, chúng ta khai báo như sau

```
\begin{itemize}
\item Nội dung thứ nhất được viết ở đây.
\item Nội dung thứ hai được viết ở đây.
\item ...
\end{itemize}
```

- Đánh số (numering) là kiểu liệt kê có thứ tự. Để sử dụng kiểu liệt kê đánh số, chúng ta khai báo như sau

```
\begin{enumerate}
\item Nội dung thứ nhất được viết ở đây.
\item Nội dung thứ hai được viết ở đây.
\item ...
\end{enumerate}
```

Hình A.1: Internet vạn vật

`\end{enumerate}`

Chú ý các nội dung trình bày trong cả hai môi trường liệt kê theo sau lệnh `\item`. Ngoài ra LaTeX còn cung cấp một số kiểu liệt kê khác, sinh viên có thể tham khảo tại <https://www.overleaf.com/learn/latex/Lists>

A.3 Cách thêm bảng

Col1	Col2	Col2	Col3
1	6	87837	787
2	7	78	5415
3	545	778	7507
4	545	18744	7560
5	88	788	6344

Bảng A.1: Table to test captions and labels.

Bảng A.1 là ví dụ về cách tạo bảng. Tất cả các bảng biểu phải được đề cập đến trong phần nội dung và phải được phân tích và bình luận. Chú ý: Tạo bảng trong LaTeX khá phức tạp và mất thời gian, vì vậy sinh viên có thể sử dụng các công cụ hỗ trợ tạo bảng (Ví dụ: <https://www.tablesgenerator.com/>). Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong LaTeX tại link <https://www.overleaf.com/learn/latex/Tables>.

A.4 Chèn hình ảnh

Hình A.1 là ví dụ về cách chèn ảnh. Lưu ý chú thích của hình vẽ được đặt ngay dưới hình vẽ. Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong LaTeX tại https://www.overleaf.com/learn/latex/Inserting_Images.

Chú ý, tất cả các hình vẽ phải được đề cập đến trong phần nội dung và phải được

phân tích và bình luận.

A.5 Tài liệu tham khảo

Cách liệt kê

Áp dụng cách liệt kê theo quy định của IEEE. Ví dụ của việc trích dẫn như sau [2]. Cụ thể, sinh viên sử dụng lệnh `\cite{}` như sau [3]. Chỉ những tài liệu được trích dẫn thì mới xuất hiện trong phần Tài liệu tham khảo. Tài liệu tham khảo cần có nguồn gốc rõ ràng và phải từ nguồn đáng tin cậy. Hạn chế trích dẫn tài liệu tham khảo từ các website, từ wikipedia.

Các loại tài liệu tham khảo

Các nguồn tài liệu tham khảo chính là sách, bài báo trong các tạp chí, bài báo trong các hội nghị khoa học và các tài liệu tham khảo khác trên internet.

A.6 Cách viết phương trình và công thức toán học

Các gói `amsmath`, `amssymb`, `amsfonts` hỗ trợ viết phương trình/công thức toán học đã được bổ sung sẵn ở phần đầu của file `main.tex`. Một ví dụ về tạo phương trình (A.1) như sau

$$F(x) = \int_b^a \frac{1}{3}x^3 \quad (\text{A.1})$$

Phương trình A.1 là ví dụ về phương trình tích phân. Một phương trình khác không được đánh số thứ tự (gán nhãn)

$$x[t_n] = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} X[f_k] e^{j2\pi nk/N}$$

Phương trình này thể hiện phép biến đổi Fourier rời rạc ngược (IDFT).

A.7 Qui cách đóng quyển

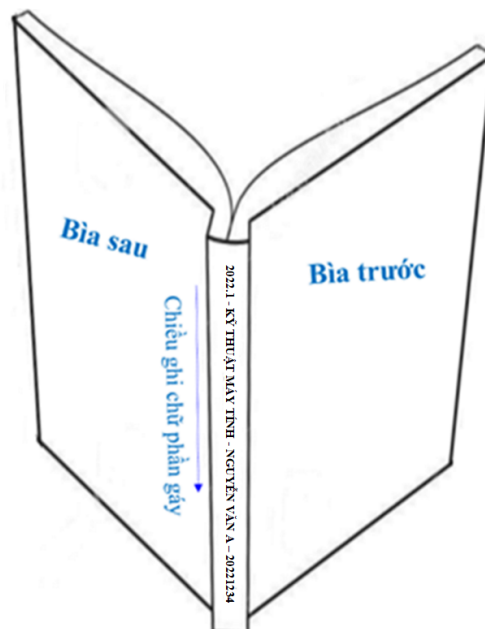
Phần bìa trước chế bản theo qui định; bìa trước và bìa sau là giấy liền khổ. Sử dụng keo nhiệt để dán gáy khi đóng quyển thay vì sử dụng băng dính và dập ghim như mô tả ở Hình A.3 Phần gáy ĐATN cần ghi các thông tin tóm tắt sau: Kỳ làm ĐATN - Ngành đào tạo - Họ và tên sinh viên - Mã số sinh viên. Ví dụ:

2022.1 - KỸ THUẬT MÁY TÍNH - NGUYỄN VĂN A - 20221234

Qui cách ghi chữ phần gáy như hình dưới đây:



Hình A.2: Qui cách đóng quyển đồ án



Hình A.3: Qui cách đóng quyển đồ án

B. ĐẶC TẢ USE CASE

Nếu trong nội dung chính không đủ không gian cho các use case khác (ngoài các use case nghiệp vụ chính) thì đặc tả thêm cho các use case đó ở đây.

B.1 Đặc tả use case “Thông kê tình hình mượn sách”

...

B.2 Đặc tả use case “Đăng ký làm thẻ mượn”

...